University of Colorado Denver

Department of Computer Science & Engineering

CSCI/CSCY 4407: Security & Cryptography

Lab 9 Assignment: Key Distribution and Trust in Cryptographic Systems: PKI, Diffie–Hellman, and TLS

Instructor: Dr. Victor Kebande

Teaching Assistant: Celest Kester

Spring 2026

---

1 Assignment Instructions

This is a group assignment. Each group is required to complete and submit its own original work. You may discuss high-level concepts related to public-key cryptography, symmetric-key cryptography, key exchange, certificate authorities, public key infrastructures, certificate chains, Diffie–Hellman key exchange, active and passive adversaries, session keys, and authentication. However, you are not permitted to share scripts, certificate files, key files, screenshots, terminal outputs, explanations, attack traces, or final report text with other groups.

This assignment combines both implementation and analysis. Some tasks require you to generate keys, create certificate signing requests, issue certificates, verify certificates, and implement a toy Diffie–Hellman exchange. Other tasks require you to reason carefully about what can go wrong when keys are distributed insecurely, why a man-in-the-middle attacker is so dangerous, why public keys must be authenticated, and why unauthenticated Diffie–Hellman is not sufficient in the presence of an active attacker.
Each group must generate and use its own:

- working directory,
- public and private keys,
- certificate files,

- certificate requests,
- CA files,
- Python scripts,
- outputs and screenshots,
- written interpretations and conclusions.

Because each group will use different names, keys, timestamps, certificate serial values, and script outputs, the exact results will differ across groups. This is intentional and forms part of the assessment design. All submitted work must reflect your group's own implementation, screenshots, analysis, and explanations.

2 Academic Integrity and Collaboration

- Allowed: discussing lecture concepts and high-level security ideas.
- Allowed: consulting Linux and OpenSSL manual pages for syntax help.
- Not allowed: sharing code, certificates, private keys, screenshots, terminal outputs, reports, or answers with other groups.
- Not allowed: copying solutions from the internet, public repositories, generative tools, or previous student work without independent implementation and interpretation.
- Submission: only one group member should submit the final report on Canvas on behalf of the group.

3 Introduction

Secure communication does not begin with encryption. It begins with trust. This is one of the most important lessons in cryptography and network security. In many introductory discussions, students are taught that confidentiality can be achieved simply by encrypting a message using a strong cryptographic algorithm. While this is partially true, it hides the deeper and more practical problem: **how do the communicating parties obtain the right key in the first place?**
Suppose Alice wants to send a confidential message to Bob. If Alice and Bob already share a symmetric key K, then Alice can encrypt a message M as

$$C \leftarrow \mathsf{Enc}_K(M)$$

and Bob can decrypt it as

$$M \leftarrow \mathsf{Dec}_K(C).$$

This looks straightforward, but it immediately raises a difficult question: **how did Alice and Bob obtain the shared secret key K?** If the key itself must be sent over the network

before communication begins, then the key distribution step becomes the weakest point of the entire system.

Now consider the public-key setting. Alice generates a public/private key pair:

$$(pk_A, sk_A) \leftarrow \mathcal{K}.$$

Bob can encrypt a message for Alice using her public key:

$$C \leftarrow \mathsf{Enc}_{pk_A}(M),$$

and Alice decrypts it using her secret key:

$$M \leftarrow \mathsf{Dec}_{sk_A}(C).$$

At first glance, this appears to solve the key distribution problem because Bob only needs Alice's public key, which is not secret. However, public availability is not the same as authenticity. Bob still needs to know that the public key he is using really belongs to Alice. This is where the real challenge appears. If an adversary Eve can intercept Alice's public key and replace it with her own public key pk_E, then Bob will unknowingly encrypt messages for Eve rather than Alice:

$$C \leftarrow \mathsf{Enc}_{pk_E}(M).$$

Eve can then decrypt the message with her secret key sk_E, learn the contents, and possibly re-encrypt the message under Alice's true public key before forwarding it. In this case, Alice and Bob may believe they are communicating securely, while in reality the attacker sits invisibly between them. This is the classic **man-in-the-middle (MITM) attack**.

The same challenge appears in key exchange protocols. One of the most elegant methods for establishing a shared secret over an insecure channel is the Diffie–Hellman key exchange. Let $G = \langle g \rangle$ be a cyclic group. Alice chooses a random secret exponent x, Bob chooses a random secret exponent y, and they exchange:

$$X = g^x, \qquad Y = g^y.$$

Alice computes

$$K_A = Y^x = g^{yx},$$

and Bob computes

$$K_B = X^y = g^{xy}.$$

Because exponentiation commutes in the exponent, both derive the same shared key:

$$K_A = K_B = g^{xy}.$$

This is a powerful idea because an eavesdropper who sees g, X, and Y should not be able to compute g^{xy} efficiently if the Computational Diffie–Hellman problem is hard in the chosen group.

However, this only protects against a **passive adversary**. If an active adversary can intercept messages and substitute new values, then Diffie–Hellman becomes vulnerable. Eve can

intercept Alice's value X, replace it with her own value, do the same with Bob's value Y, and establish two separate shared keys:

$$K_{AE} \quad \text{between Alice and Eve,} \qquad K_{EB} \quad \text{between Eve and Bob.}$$

Alice believes she shares a key with Bob. Bob believes he shares a key with Alice. In fact, both share keys only with Eve. This shows that key exchange without authentication is not enough in the presence of an active attacker.

To solve these authenticity problems in practice, modern systems use a **Public Key Infrastructure (PKI)**. In a PKI, a trusted entity called a **Certificate Authority (CA)** signs data that binds an identity to a public key. A certificate can be viewed conceptually as

$$CERT_A = (\mathrm{ID}_A, pk_A, \sigma_{CA}),$$

where ID_A is Alice's identity information, pk_A is Alice's public key, and σ_{CA} is a signature produced using the CA's secret key. Any verifier who trusts the CA's public key can verify the certificate:

$$\mathsf{Verify}_{pk_{CA}}(CERT_A) = 1.$$

If the certificate is valid and the verifier trusts the CA, then the verifier gains confidence that the public key contained in the certificate truly belongs to the named party.

Certificates are useful because they shift the trust problem. Bob no longer needs to know Alice personally or receive her public key through a secure side channel. Instead, Bob only needs to trust the CA or a chain of CAs leading back to a trusted root certificate already installed in the operating system or browser. This is the basis of trust in protocols such as TLS.

Even then, PKI is not perfect. Certificates can expire, be mis-issued, or need to be revoked if a private key is compromised. Certificate revocation itself introduces delays and management challenges. These practical issues remind us that key distribution is not a trivial setup step; it is a core security problem that affects the confidentiality, integrity, and authenticity of the entire communication process.

In this lab, you will study the key distribution problem from both practical and conceptual perspectives. You will first observe how trust fails when a public key is substituted. You will then construct a small certificate infrastructure using OpenSSL by generating a CA, creating a certificate signing request, issuing a certificate, and verifying it. After that, you will implement a toy Diffie–Hellman exchange in Python, observe why it works under passive observation, and then reason carefully about why it fails under active attack. Finally, you will connect all these ideas to the notion of session keys and authenticated key exchange used in real-world systems such as TLS, SSL, SSH, and IPsec (Read what is IPSec in brief).

The goal of this lab is not only to make the mechanisms work, but to make you understand why they are needed, what security assumptions they rely on, and how those assumptions fail when authenticity is missing.

4 Motivating Case Scenario

Case Scenario: When Encryption Is Not Enough

A financial department at a medium-sized organization wants to send confidential payroll data from Alice, a payroll officer, to Bob, a systems administrator responsible for secure archival. Bob asks Alice to send her public key so that he can encrypt the payroll file before transmission. Alice replies with what she believes is her public key, but an active attacker Eve is monitoring the network. Eve intercepts Alice's public key and substitutes her own public key before the message reaches Bob. Bob, believing that the key belongs to Alice, encrypts the payroll file and sends it. Eve decrypts the file, reads the confidential contents, optionally modifies them, and then re-encrypts the file using Alice's actual public key before forwarding it. Bob believes he communicated securely with Alice. Alice receives a properly encrypted file and may not realize that confidentiality was already lost, this has been shown in Figure 1.

Later, the organization attempts to improve its security by deploying a key exchange protocol so that Alice and Bob can establish a shared session key before transmitting sensitive data. They implement a basic Diffie–Hellman exchange, but they do not authenticate the exchanged values. Eve again intercepts the messages, substitutes her own values, and establishes one shared key with Alice and another with Bob. Both legitimate parties believe they share one secure session key with each other, while in fact Eve sits invisibly between them and can decrypt, inspect, alter, and re-encrypt all traffic, this is also shown in Figure 2.

This scenario illustrates the central theme of this lab: strong cryptographic algorithms do not automatically provide secure communication. The real challenge is establishing keys *authentically*. This lab will guide you through the technical and conceptual steps needed to understand why key distribution is difficult, why certificate-based trust exists, why unauthenticated Diffie–Hellman is insufficient against active attackers, and why modern protocols use authenticated session-key establishment.

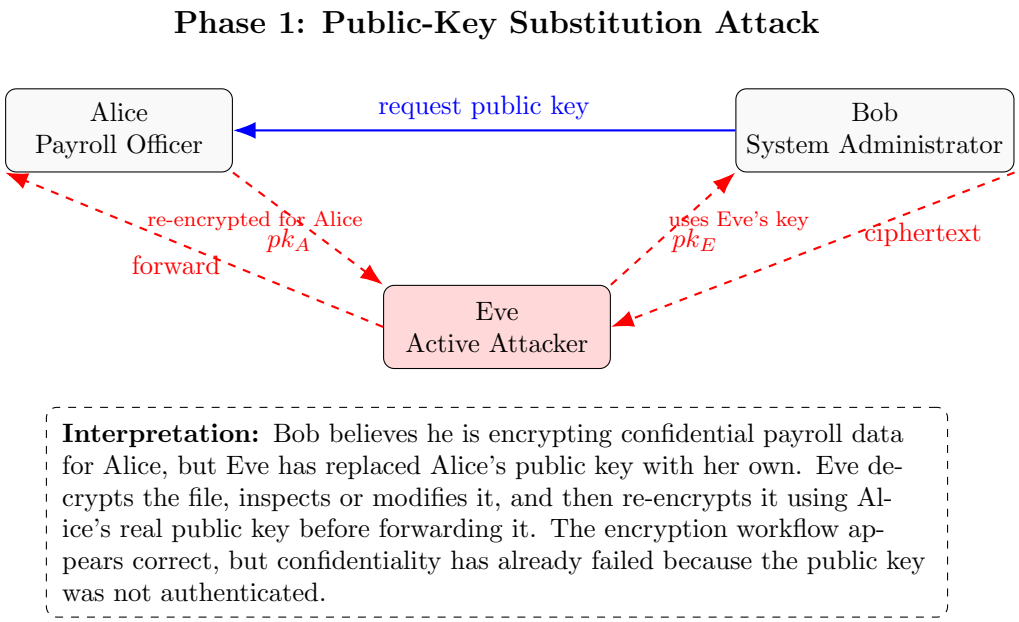


Figure 1: Public-key substitution attack in the motivating case scenario.

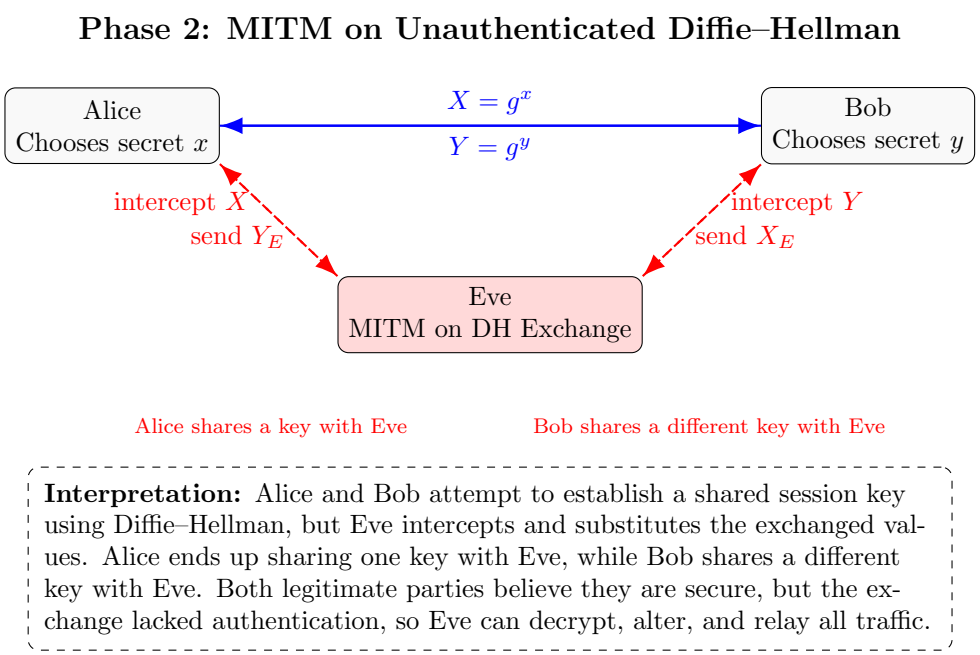


Figure 2: MITM attack on unauthenticated Diffie–Hellman in the motivating case scenario.

5 Assignment Objectives

The objective of this assignment is to develop a practical and theoretical understanding of key distribution in both public-key and shared-key settings. By the end of the lab, students

should be able to connect lecture-level concepts about trust, authenticity, certificates, and key exchange to actual commands, scripts, outputs, and attack scenarios.
By completing this assignment, students will be able to:

- Explain why key distribution is a fundamental problem in cryptography.
- Distinguish between secrecy of a key and authenticity of a key.
- Explain the danger of man-in-the-middle attacks in both public-key delivery and Diffie–Hellman exchange.
- Generate RSA key pairs for a user and for a certificate authority.
- Create a certificate signing request and understand its role.
- Issue a certificate using a CA and explain what is being certified.
- Verify certificates and interpret the result correctly.
- Explain why trust must be anchored in a known CA public key.
- Implement a toy Diffie–Hellman key exchange in Python.
- Explain why Diffie–Hellman is secure against passive eavesdropping but not against active attack.
- Distinguish long-lived keys from session keys.
- Relate the lab concepts to real-world systems such as TLS and PKI-based communication.

6 Overview of Lab Tasks

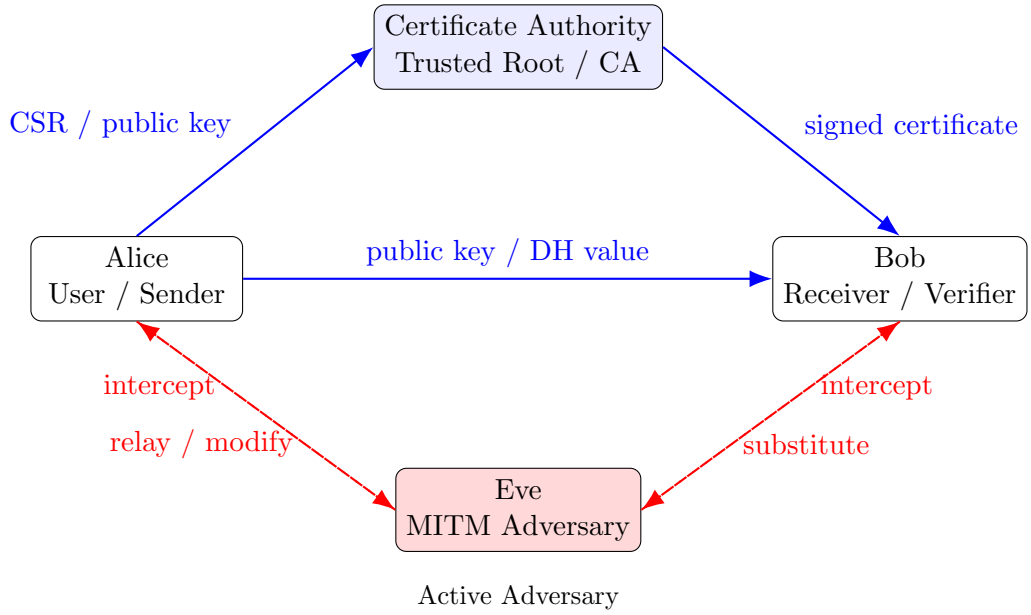
Task	Title	Primary Learning Goal
Task 1	Working Directory and Message Setup	Prepare the lab environment and create files used in the experiments.
Task 2	Public Key Substitution Scenario	Understand how authenticity fails when a public key is replaced.
Task 3	User Key Generation	Generate a public/private key pair for a user.
Task 4	Certificate Authority Setup	Generate a CA key pair and self-signed root certificate.
Task 5	Certificate Signing Request	Create a CSR and examine its purpose.
Task 6	Certificate Issuance	Use the CA to sign a user certificate.
Task 7	Certificate Verification	Verify a certificate and understand trust anchoring.
Task 8	Wrong Trust Anchor / Fake CA Experiment	Show that trust depends on which CA public key is trusted.
Task 9	Toy Diffie–Hellman Implementation	Implement shared-key agreement in Python.
Task 10	Passive Adversary Analysis	Explain why observing exchanged values is not enough to recover the key.
Task 11	Active MITM Attack on Diffie–Hellman	Show why unauthenticated DH fails under active attack.
Task 12	Session Keys and Final Reflection	Explain long-lived vs session keys and reflect on the lab.

7 Conceptual Lab Diagram

The conceptual diagram shown in Figure 3 illustrates the fundamental challenge addressed in this lab: establishing cryptographic keys securely in the presence of an active adversary. Alice and Bob represent two legitimate parties attempting to communicate securely as we have seen in the motivating scenario, while Eve represents a man-in-the-middle (MITM) attacker capable of intercepting, modifying, and relaying messages. The blue arrows indicate the intended secure flow of information, where Alice provides her public key or key exchange value, and Bob uses it to establish confidentiality. However, the red dashed arrows demonstrate how this process can be subverted when authenticity is not guaranteed. Eve can intercept Alice's keying material and substitute it with her own, causing Bob to unknowingly establish a secure channel with the attacker instead of Alice.

To address this vulnerability, the diagram introduces a Certificate Authority (CA), which acts as a trusted root for binding identities to public keys. Rather than relying on unauthenticated key exchange, Bob verifies Alice's public key through a certificate signed by the CA. This ensures that the key genuinely belongs to Alice and has not been altered in transit. The diagram therefore contrasts two scenarios: an insecure setting where key distribution is

vulnerable to MITM attacks, and a secure setting where trust is established through certification. This highlights a central insight of modern cryptography: strong encryption alone is insufficient, secure communication depends critically on authenticated key distribution.



Interpretation: Without authentication, Eve can replace Alice's public key or Diffie–Hellman value and establish two separate channels. With PKI, Bob verifies Alice's certified public key using a trusted CA certificate. The lab explores both the failure case and the defended case.

Figure 3: Conceptual diagram for key distribution and MITM attack.

8 Lab Requirements

To successfully complete this lab, students must prepare a Linux-based environment suitable for certificate generation, certificate verification, and Python scripting. The recommended platforms are:

- Ubuntu Linux
- Kali Linux
- A university lab workstation running Linux
- A Linux virtual machine using VirtualBox or VMware

8.1 Required Software Tools

The following tools are recommended:

- OpenSSL – used to generate keys, create CSRs, issue certificates, and verify certificates.
- Python 3 – used for toy Diffie–Hellman implementation and simple reasoning scripts.
- Linux terminal – used to execute commands and capture outputs.
- Text editor or code editor – such as nano, vim, gedit, or VS Code.

8.2 Required Installations

Install the required tools if they are not already available:

```
sudo apt update
sudo apt install openssl python3 python3-pip
```

Verify the installations:

```
openssl version
python3 --version
```

8.3 Working Directory Setup

Create a dedicated working folder for this lab:

```
mkdir Key_Distribution_Lab
cd Key_Distribution_Lab
```

All key files, certificate files, scripts, screenshots, notes, and outputs should be stored in this directory.

8.4 Documentation Requirements

Your final report must include:

- screenshots showing executed commands and outputs,
- evidence of key generation,
- evidence of certificate issuance and verification,
- explanations of what the commands are doing,
- Python scripts used in the Diffie–Hellman tasks,
- clear written explanations of observations and attack results,
- a final comparison and reflection.

9 Lab Tasks

This lab is completed step by step. Read each task carefully before implementation. Do not skip ahead. Several later tasks depend on files created in earlier tasks.

9.1 Task 1: Create the Working Directory and Test Files

Begin by creating the working directory and preparing a few sample message files. These files will not be the core of the certificate workflow, but they provide concrete artifacts that help you reason about which party is sending what, which key is supposed to be used, and what the attacker gains if trust fails.

Step 1: Create the directory

```
mkdir Key_Distribution_Lab
cd Key_Distribution_Lab
```

Step 2: Create message files

```
echo "Confidential payroll report for Alice." > msg1.txt
echo "Server backup credentials for secure transfer." > msg2.txt
echo "Session initialization data for Bob." > msg3.txt
```

Step 3: Display the files

```
cat msg1.txt
cat msg2.txt
cat msg3.txt
```

Step 4: Inspect the working directory

```
pwd
ls -l
```

Step 5: Compute hashes of the files

```
sha256sum msg1.txt msg2.txt msg3.txt
```

What to do:

- Create the working directory.
- Create three plaintext files.
- Display the file contents.
- Inspect the directory contents.
- Compute SHA-256 hashes for all three files.
- Include screenshots showing the setup and file creation.

What to explain:

- Why it is useful to have concrete files when reasoning about key misuse.
- Why the content of a protected message matters if an attacker receives the wrong key.

9.2 Task 2: Study the Public Key Substitution Problem Conceptually (Use pen and paper for the workflow and show proof for correctness, everything in this task is pen and paper)

In this task, you will not begin with a command. Instead, you will first reason about the core authenticity problem. The goal is to show that public-key encryption does not solve key distribution automatically. It only shifts the problem from secret distribution to authentic public-key delivery.
Assume the following intended scenario:

- Alice generates (pk_A, sk_A).
- Bob wants to send Alice a confidential message.
- Bob requests Alice's public key.
- Eve intercepts the reply and sends Bob her own public key pk_E instead.

Step 1: Write the intended secure flow

$$(pk_A, sk_A) \leftarrow \mathcal{K}$$
$$C \leftarrow \mathsf{Enc}_{pk_A}(M)$$
$$M \leftarrow \mathsf{Dec}_{sk_A}(C)$$

Step 2: Write the attacked flow

$$(pk_E, sk_E) \leftarrow \mathcal{K}$$
$$C \leftarrow \mathsf{Enc}_{pk_E}(M)$$
$$M \leftarrow \mathsf{Dec}_{sk_E}(C)$$

Step 3: In your report, describe the message flow step by step Your description should include:

- What Bob believes he is doing.
- What Eve is actually causing Bob to do.
- What Eve learns.
- Whether Alice and Bob necessarily detect the attack.

What to do:

- Write a one-paragraph description of the intended secure exchange.
- Write a one-paragraph description of the attacked exchange.
- Explain why public availability of a key is not the same as authenticity of a key.

What to explain:

- Why confidentiality fails even though Bob used "a public key" correctly.
- Why the failure is not caused by weak encryption but by missing authenticity.

9.3 Task 3: Generate a Public/Private Key Pair for Alice

You will now generate an RSA key pair for Alice. This gives you an actual user key pair that will later be certified by a certificate authority.

Step 1: Generate Alice's private key

```
openssl genpkey -algorithm RSA -out alice_private.pem -pkeyopt rsa_keygen_bits:2048
```

Step 2: Extract Alice's public key

```
openssl rsa -pubout -in alice_private.pem -out alice_public.pem
```

Step 3: Inspect the resulting files

```
ls -l alice_private.pem alice_public.pem
```

Step 4: Optionally inspect the key details

```
openssl pkey -in alice_private.pem -text -noout
openssl rsa -pubin -in alice_public.pem -text -noout
```

What to do:

- Generate a 2048-bit RSA private key for Alice.
- Extract the matching public key.
- Record screenshots showing the commands and resulting files.
- Briefly explain which key must remain secret and which key may be distributed.

What to explain:

- Why Bob still cannot simply trust a public key file unless it is authenticated.
- Why generating a key pair alone does not solve the key distribution problem.

9.4 Task 4: Create a Certificate Authority (CA)

In practice, a public key becomes more trustworthy when a trusted authority signs information binding that key to an identity. In this task, you will create a simple CA. For educational purposes, your CA will be self-created and self-signed. In real systems, root CAs are trusted because their public keys are already embedded in browsers and operating systems.

Step 1: Generate the CA private key

```
openssl genpkey -algorithm RSA -out ca_private.pem -pkeyopt rsa_keygen_bits:2048
```

Step 2: Create a self-signed root certificate

```
openssl req -x509 -new -key ca_private.pem -out ca_cert.pem -days 3650
```

When prompted, enter meaningful values such as:

- Country Name
- State or Province
- Organization Name
- Common Name (for example: Lab Root CA)

Step 3: Inspect the CA certificate

```
openssl x509 -in ca_cert.pem -text -noout
```

Step 4: List the files

```
ls -l ca_private.pem ca_cert.pem
```

What to do:

- Generate a private key for the CA.
- Create a self-signed CA certificate.
- Inspect the certificate fields.
- Record screenshots showing the commands and certificate output.

What to explain:

- What a CA is supposed to do conceptually.
- Why trust in a CA must be anchored somewhere.
- Why the CA certificate is self-signed in this lab but externally trusted in real systems.
- Why do you use rsa 2048?

9.5 Task 5: Create a Certificate Signing Request (CSR) for Alice

A certificate signing request is how a user asks a CA to certify a public key. It typically contains the user's identity details and public key, and it is associated with proof that the requester controls the private key.

Step 1: Create Alice's CSR

```
openssl req -new -key alice_private.pem -out alice.csr
```

When prompted, provide identity information for Alice. Make the Common Name clearly identifiable, for example:

```
Alice Student
```

Step 2: Inspect the CSR

```
openssl req -in alice.csr -text -noout
```

Step 3: List the CSR file

```
ls -l alice.csr
```

What to do:

- Create a CSR for Alice using Alice's private key.
- Inspect the contents of the CSR.
- Include screenshots showing the request and the inspected output.

What to explain:

- What information is included in a CSR.
- Why a CA does not simply sign an arbitrary public key without identity checks.
- Why proof of possession of the private key matters.

9.6 Task 6: Issue a Certificate for Alice Using the CA

Now that Alice has generated a CSR, the CA can issue a certificate. This is the step where the CA signs information associating Alice's identity with Alice's public key as is shown in Figure 4.

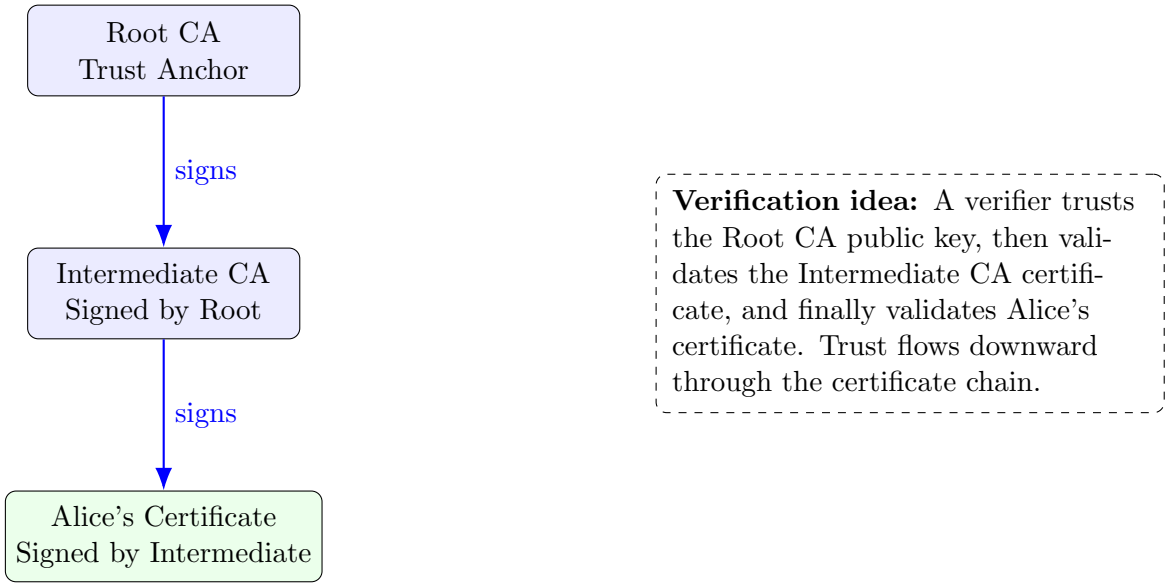


Figure 4: Example certificate chain: trust begins at the root and extends through signed certificates.

Step 1: Sign Alice's CSR using the CA

```
openssl x509 -req -in alice.csr -CA ca_cert.pem -CAkey ca_private.pem \
-CAcreateserial -out alice_cert.pem -days 365 -sha256
```

Step 2: Inspect Alice's certificate

```
openssl x509 -in alice_cert.pem -text -noout
```

Step 3: List the resulting files

```
ls -l alice_cert.pem ca_cert.srl
```

Step 4: Compare issuer and subject fields Look carefully in the certificate text output:

- The **Subject** should correspond to Alice.
- The **Issuer** should correspond to the CA.

What to do:

- Use the CA to sign Alice's CSR.
- Inspect the issued certificate.
- Show that the subject and issuer fields differ appropriately.
- Include screenshots of the commands and outputs.

What to explain:

- What exactly the CA is asserting when it signs Alice's certificate.
- Why the certificate is more useful than sending a raw public key file by email.
- Why this process helps Bob trust the key if Bob already trusts the CA.

9.7 Task 7: Verify Alice's Certificate

A certificate is only useful if a verifier can check it using a trusted CA certificate. In this task, you will verify Alice's certificate against the CA certificate created earlier.

Step 1: Verify the certificate

```
openssl verify -CAfile ca_cert.pem alice_cert.pem
```

A successful result should indicate that the certificate is valid under the CA file supplied.

Step 2: Extract certificate subject and issuer if needed

```
openssl x509 -in alice_cert.pem -noout -subject -issuer
```

Step 3: Inspect validity dates

```
openssl x509 -in alice_cert.pem -noout -dates
```

What to do:

- Verify Alice's certificate using the CA certificate.

- Record the terminal output showing the verification result.
- Inspect the subject, issuer, and validity period.
- Include screenshots showing each step.

What to explain:

- Why verification only makes sense relative to a trusted CA certificate.
- Why a certificate being mathematically valid is not enough if the wrong CA is trusted.
- Why certificate validity dates matter.

6.8 Task 8: Wrong Trust Anchor / Fake CA Experiment

One of the most important ideas in PKI is that trust depends on which root or CA public key you accept. In this task, you will create a fake CA and show that trust can be made to "work" under the wrong trust anchor.

Step 1: Generate a fake CA private key

```
openssl genpkey -algorithm RSA -out fake_ca_private.pem -pkeyopt rsa_keygen_bits:2048
```

Step 2: Create a fake CA certificate

```
openssl req -x509 -new -key fake_ca_private.pem -out fake_ca_cert.pem -days 3650
```

Step 3: Generate a fake user key pair

```
openssl genpkey -algorithm RSA -out eve_private.pem -pkeyopt rsa_keygen_bits:2048
openssl rsa -pubout -in eve_private.pem -out eve_public.pem
```

Step 4: Create a CSR for Eve

```
openssl req -new -key eve_private.pem -out eve.csr
```

Step 5: Sign Eve's CSR with the fake CA

```
openssl x509 -req -in eve.csr -CA fake_ca_cert.pem -CAkey fake_ca_private.pem \
-CAcreateserial -out eve_cert.pem -days 365 -sha256
```

Step 6: Verify Eve's certificate using the fake CA

```
openssl verify -CAfile fake_ca_cert.pem eve_cert.pem
```

Step 7: Try verifying Eve's certificate using the real CA

```
openssl verify -CAfile ca_cert.pem eve_cert.pem
```

What to do:

- Create a fake CA.

- Use the fake CA to issue a certificate.
- Verify the fake certificate under the fake CA.
- Attempt verification under the original CA and observe the failure.
- Include screenshots of both success and failure.

What to explain:

- Why a certificate can appear valid under one trust anchor but invalid under another.
- Why the CA public key that is trusted is the real security anchor.
- How this experiment connects to the problem of malicious or substituted trust roots.

9.8 Task 9: Implement a Toy Diffie–Hellman Key Exchange in Python

You will now implement a small educational Diffie–Hellman example using toy values. This is not for real deployment; it is to help you see how both parties derive the same shared secret mathematically.

Step 1: Create the script

```
nano dh_demo.py
```

Use the following script:

```
p = 23
g = 5

# Alice's secret exponent
x = 6

# Bob's secret exponent
y = 15

# Public values
X = pow(g, x, p)
Y = pow(g, y, p)

# Shared keys
KA = pow(Y, x, p)
KB = pow(X, y, p)

print("Public prime p =", p)
print("Generator g =", g)
print("Alice secret x =", x)
```

```
print("Bob secret y =", y)
print("Alice sends X = g^x mod p =", X)
print("Bob sends Y = g^y mod p =", Y)
print("Alice computes KA = Y^x mod p =", KA)
print("Bob computes KB = X^y mod p =", KB)
print("Same key?", KA == KB)
```

Step 2: Run the script

```
python3 dh_demo.py
```

Step 3: Change the private exponents Modify x and y to different values and run the script again.
What to do:

- Implement the Python script.
- Run it and record the output.
- Repeat with at least one different choice of exponents.
- Include screenshots showing the script and outputs.

What to explain:

- Why Alice and Bob obtain the same key.
- Why the exchanged public values are not enough by themselves to reveal the private exponents.
- Why this example is educational and not suitable for real deployment.

9.9 Task 10: Analyze Diffie–Hellman Against a Passive Adversary (Everything should be Pen and Paper for this Task (You will lose points if you generate anything other than pen and paper)

A passive adversary only observes communication and does not modify messages. In this task, you will explain why Diffie–Hellman can still be secure in this model.
Suppose an eavesdropper sees:

$$p,\ g,\ X = g^x,\ Y = g^y.$$

Alice computes:

$$K_A = Y^x = g^{yx},$$

and Bob computes:

$$K_B = X^y = g^{xy}.$$

Step 1: In your report, describe what the attacker learns The attacker learns the public parameters and the exchanged public values.
Step 2: Explain what the attacker wants The attacker wants to compute:

$$g^{xy}.$$

Step 3: Explain the hardness assumption Explain that the attacker is assumed not to be able to efficiently derive the shared key from the observed public values if the relevant Diffie–Hellman problem is hard in the chosen group.
What to do:

- Write a paragraph describing the passive adversary model.
- Write a paragraph explaining why the shared secret remains hidden under this model.
- Relate your explanation to the outputs of your Python script.

What to explain:

- Why observing X and Y is different from knowing x and y.
- Why Diffie–Hellman is useful against eavesdropping but not yet authenticated.

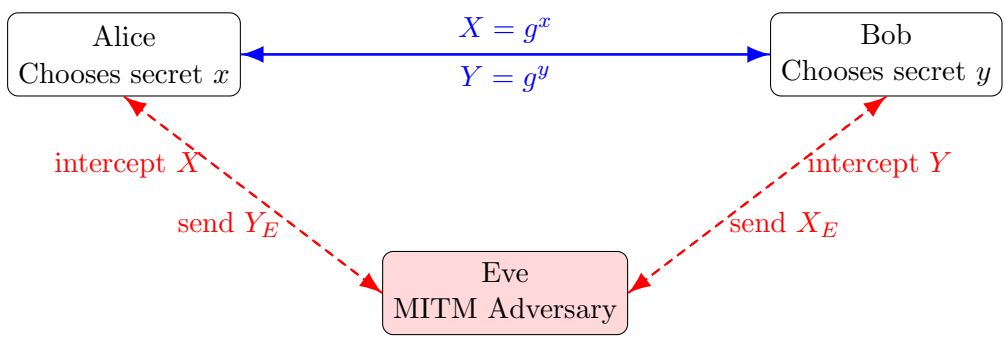
9.10 Task 11: Explain an Active MITM Attack on Diffie–Hellman (Pen and Paper)

This is one of the most important tasks in the lab. Diffie–Hellman is elegant, but without authentication, an active attacker can substitute values and create two separate keys as is shown in Figure 5.
Assume:

- Alice chooses secret x and sends $X = g^x$.
- Bob chooses secret y and sends $Y = g^y$.
- Eve chooses her own secret exponents and intercepts both exchanges.

Step 1: Describe the attacked flow For example:

- Alice sends X toward Bob.
- Eve intercepts X and sends X_E to Bob instead.
- Bob sends Y toward Alice.
- Eve intercepts Y and sends Y_E to Alice instead.



Active substitution attacker

Interpretation: Alice computes a key using Eve's substituted value Y_E, while Bob computes a key using Eve's substituted value X_E. As a result, Alice shares a key with Eve and Bob shares a different key with Eve. Neither Alice nor Bob necessarily detects the attack unless the exchanged values are authenticated.

Figure 5: Man-in-the-middle attack on unauthenticated Diffie–Hellman key exchange.

Step 2: Write the resulting keys Alice computes a key with Eve:

$$K_{AE} = (Y_E)^x$$

Bob computes a key with Eve:

$$K_{EB} = (X_E)^y$$

Step 3: Explain the deception

- Alice believes the key is shared with Bob.
- Bob believes the key is shared with Alice.
- Eve actually shares one key with Alice and a different key with Bob.

Step 4: Explain what Eve can do

- Decrypt messages from Alice.
- Read or modify them.
- Re-encrypt them for Bob.
- Repeat in the reverse direction.

What to do:

- Write a clear step-by-step attack narrative.

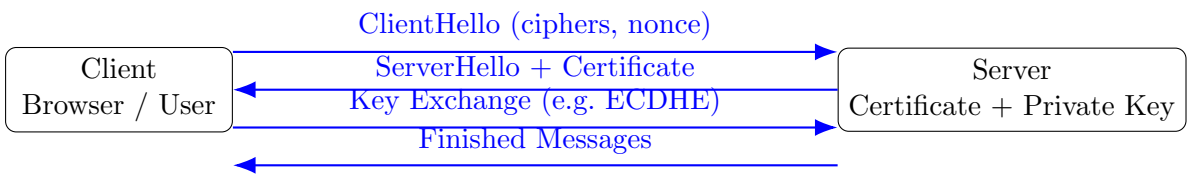
- Include the key expressions for the two attacker-mediated keys.
- Explain why the attack is possible.
- Explain why simple Diffie–Hellman does not tell Alice who sent the value she received.

What to explain:

- Why this is an authenticity failure rather than a failure of the exponentiation itself.
- Why authenticated key exchange protocols are necessary in practice.

9.11 Task 12: Session Keys, Long-Lived Keys, and Final Reflection

Real systems rarely use long-lived keys directly for large amounts of data. Instead, they use long-lived credentials to establish fresh session keys as is shown in Figure 6.



Interpretation: TLS combines certificate-based authentication with key exchange to establish a fresh session key. The certificate authenticates the server, while the handshake establishes a short-lived session key for secure communication.

Figure 6: TLS handshake: step-by-step authenticated key exchange and session establishment.

Step 1: Explain long-lived keys Examples include:

- a user's RSA private key,
- a server certificate private key,
- a trusted CA root key,
- a long-term shared secret in a specialized system.

Step 2: Explain session keys A session key is a fresh key established for a particular communication session. It is often used only for a short period.
Step 3: Explain why session keys are preferable Discuss points such as:

- efficiency,

- limiting damage if a session key is exposed,
- separation between authentication and bulk encryption,
- better protocol design.

Step 4: Write a final reflection paragraph Your final reflection should address:

- Why key distribution is harder than it first appears.
- Why public-key cryptography still needs authenticity.
- Why PKI helps.
- Why Diffie–Hellman is insufficient without authentication.
- Why modern systems use authenticated session-key establishment.

What to do:

- Write one short comparison table contrasting raw public-key delivery, certificate-based delivery, unauthenticated DH, and authenticated session-key establishment.
- Write one reflection paragraph of approximately one page or less.

10 Submission Requirements

- Submit a single PDF report on Canvas by the stated deadline. Only one group member should submit on behalf of the group.
- Your report must be clearly structured and organized by task.
- For each task, your report must include:
  - a clear explanation of what was implemented or analyzed,
  - screenshots showing execution in the Ubuntu/Kali Linux terminal where applicable,
  - certificate and key outputs where relevant,
  - Python code screenshots or listings where applicable,
  - interpretation of the results, not just raw output.
- Your Python source code must be included either as embedded code in the report or as separate files if instructed by the instructor.
- All results must be reproducible. Missing screenshots, unclear code, or undocumented outputs may result in reduced points.

11 Task-Specific Requirements

- **Task 1:** Evidence of directory creation, file creation, file display, and hashes.
- **Task 2:** Clear written description of the public-key substitution scenario.
- **Task 3:** Evidence of Alice key generation and explanation of public/private roles.
- **Task 4:** Evidence of CA key generation and self-signed certificate creation.
- **Task 5:** Evidence of CSR generation and explanation of its purpose.
- **Task 6:** Evidence of certificate issuance and explanation of the issuer/subject relationship.
- **Task 7:** Evidence of certificate verification and explanation of trust anchoring.
- **Task 8:** Evidence of fake CA setup, verification success under the fake CA, and failure under the original CA.
- **Task 9:** Python script and outputs demonstrating matching shared keys.
- **Task 10:** Passive-adversary explanation grounded in the DH exchange.
- **Task 11:** Step-by-step MITM explanation on unauthenticated DH.
- **Task 12:** Comparison table and final reflection.

12 Grading Rubric (Total: 100 Points)

Assessment Criteria	**Points**
Correct working directory setup, test files, and hash evidence	6
Clear explanation of the public-key substitution problem	8
Correct generation of Alice's RSA key pair and explanation	8
Correct generation of CA key and self-signed CA certificate	10
Correct CSR generation and explanation of purpose	8
Correct certificate issuance and interpretation of issuer/subject roles	10
Correct certificate verification and trust-anchor explanation	10
Fake CA experiment and explanation of trust dependence	10
Correct toy Diffie–Hellman implementation and outputs	8
Passive adversary analysis and explanation of DH security intuition	6
Active MITM explanation on Diffie–Hellman	8
Session-key discussion, comparison table, and final reflection	8
Report quality: clarity, completeness, screenshots, organization, and depth of explanation	10
Total	**100**

13 Notes to Students

This lab contains both practical commands and conceptual reasoning. Some tasks use real tools such as OpenSSL to create keys and certificates, while other tasks use small educational examples to help you understand how shared secret establishment works and why it fails without authentication. The toy Diffie–Hellman example is intentionally small and insecure for real-world deployment. It is included only for learning.
Your report should clearly distinguish between:

- what is practical in real systems,
- what is simplified for educational purposes,
- what security property is being achieved,

- and what assumption is required for that property to hold.